Professional Self Assessment

Alondra Paulino Santos
CS 499 Computer Science Capstone
October 17, 2026

Professional Identity and Career Direction

My interest in computing began long before I understood that it could become a career. In 2003, my family had a Gateway desktop computer with a large monitor and an internet connection that shared the telephone line. When I was online, my mother sometimes had to ask me to disconnect so she could make a phone call. That early experience made computers feel exciting and full of possibilities. During my teenage years, I continued learning how computers worked and built my first computer. Those experiences created a lasting interest in technology, even though my path into college and computer science was not immediate.

Life circumstances prevented me from attending college directly after high school. Years later, the disruption caused by the COVID-19 pandemic gave me an opportunity to reconsider the direction of my career. I began exploring technology through a course and quickly realized how much more I wanted to learn. I enrolled in an information technology program because I initially believed my strongest interest was in computer hardware, systems, and the visible parts of technology. During the second year of that program, I took a Python programming course. Writing code changed my understanding of what I could create with a computer. I became interested not only in how a system operates, but also in how software logic, data, and interfaces work together to solve a problem. After completing that course and researching the field, I transferred into computer science because it aligned more closely with the work I wanted to pursue.

My professional goal is to work in backend or full-stack software development. I am especially interested in building reliable applications, APIs, databases, and authentication systems that support useful user experiences. The Computer Science program and CS 499 helped me move from completing isolated programming assignments to evaluating software as a complete system. I now consider maintainability, data integrity, performance, security, testing, and the needs of different users when making a technical decision. This portfolio demonstrates that growth through three enhanced artifacts in software design and engineering, algorithms and data structures, and databases.

Growth Through the Computer Science Program

My education has changed both my technical abilities and the way I approach unfamiliar problems. Early programming courses taught me how to decompose a problem, trace logic, use functions and classes, and test whether a solution behaves as expected. Later courses required me to work across application layers, select data structures, connect applications to databases, and explain the reasoning behind design choices. Coursework outside the three portfolio artifacts also expanded my perspective. For example, studying cloud and serverless computing helped me

think about scalability, deployment, availability, cost, privacy, and vendor dependence. Exploring augmented and mixed reality showed me that even highly visual applications depend on secure APIs, databases, authentication, and dependable backend services.

CS 499 brought these areas together through code review, iterative enhancement, evidence collection, and reflection. Instead of treating an application as finished because it runs, I learned to ask whether its inputs are validated, whether errors are handled safely, whether a design can scale, whether an algorithm is appropriate, and whether another developer or stakeholder can understand the result. Instructor feedback also became part of the engineering process. I used it to strengthen implementation evidence, clarify architectural responsibilities, add testing results, document efficiency with Big-O analysis, and connect technical work more explicitly to the course outcomes.

Collaboration and Decision Making

The first course outcome focuses on collaborative environments and decision making for diverse audiences. Although the capstone artifacts were individual projects, the work reflects collaborative practices used in professional software development. My code review identified limitations in the original applications and communicated a plan for improvement. I organized findings by software design, algorithms, databases, and security so that another developer could understand the technical concerns and so that a nontechnical stakeholder could understand why the changes mattered. Contextual comments, descriptive names, validation messages, test evidence, and design documentation make the enhanced work easier to review and maintain.

I also treated instructor feedback as a form of stakeholder input. For the Travlr Getaways application, I documented the exact validation, authentication, API-security, and testing changes rather than describing them only in general terms. For the course-planning application, I implemented the proposed algorithm, tested successful and unsuccessful dependency scenarios, and explained its efficiency. For the animal-rescue dashboard, I clarified which responsibilities belong to the interface, data-access layer, and database. This process demonstrates that collaboration is not limited to dividing tasks among team members. It also includes listening to review findings, explaining trade-offs, preserving evidence, and revising a solution so that others can make informed decisions.

Professional Communication

The second course outcome requires professional oral, written, and visual communication adapted to specific audiences and contexts. My code-review video explains the condition of the original artifacts and the enhancements I planned. The enhancement narratives describe the original limitations, the implemented changes, the testing process, and the connection to the course outcomes. Updated diagrams and interface screenshots communicate application behavior visually, while tables and performance measurements allow a reader to evaluate results without relying on unsupported claims.

I have also learned to adjust the level of technical detail to the audience. A developer needs information about validation rules, query construction, algorithm complexity, and test conditions.

An administrator or customer needs clear controls, useful status information, and error messages that explain what action to take without exposing sensitive details. My portfolio uses both forms of communication. The source code and technical narratives support developer review, while the interfaces, summaries, and evidence explain the value of the improvements to a broader audience.

Algorithmic Problem Solving and Trade Offs

The third course outcome is demonstrated most directly by the enhanced CS 300 course-planning application. The original program stored and retrieved course information, but the enhancement treats prerequisites as a directed graph. An adjacency-list representation models the dependency relationships, and Kahn's topological-sort algorithm generates a prerequisite-respecting order. A queue holds courses whose current in-degree is zero, allowing the program to process courses only after their prerequisites have been satisfied. The program also detects circular dependencies, missing prerequisites, self-prerequisites, duplicate courses, duplicate prerequisite entries, malformed data, and invalid course codes.

This design required explicit trade-offs. An adjacency matrix could make an individual edge lookup direct, but it would use quadratic space even when the prerequisite graph is sparse. The adjacency-list approach uses $O(V + E)$ space, where V is the number of courses and E is the number of prerequisite relationships. Loading and validation are expected to run in $O(V + E)$ time on average with hash-based lookup, and topological sorting also runs in $O(V + E)$ time. The implementation therefore provides predictable performance while preserving readable validation output. Tests with valid input, dependency cycles, missing references, self-dependencies, duplicate records, malformed records, strict compiler warnings, and sanitizers provide evidence that the solution handles both expected and adverse conditions.

Software Engineering Database Design and Industry Value

The fourth course outcome requires well-founded techniques, skills, and tools that deliver practical value. The enhanced CS 465 Travlr Getaways application demonstrates this through a complete customer-inquiry workflow and supporting administrative tools. Customers can search and filter trips, submit a validated inquiry, and receive a reference number. Administrators can authenticate, review inquiries, filter by status, and update progress. The enhancement also corrected price handling, strengthened input validation, protected API operations, improved error handling, and expanded testing. These changes move the application beyond a basic classroom demonstration by addressing workflows, maintainability, and data consistency that a real organization would need.

The enhanced CS 340 animal-rescue dashboard demonstrates the relationship between application design and database performance. Server-side filtering, pagination, counting, aggregation, and deterministic sorting place data-intensive work in MongoDB instead of loading and processing an entire result set in the interface. Query allow-listing restricts supported fields and operators before queries reach the database. The application also validates requests, reports failures safely, and separates presentation responsibilities from authoritative data-access controls.

Automated tests passed across the enhancement, and measurements using 10,000 records compared query execution before and after indexing. The index improved the target read pattern, while the narrative also recognizes its storage and write-maintenance costs. That evidence-based trade-off is important because an industry-standard solution should justify a design decision rather than assume that every optimization is beneficial.

Security Mindset

The fifth course outcome changed how I think about application design. Security is not a feature that can be added only after an application is complete. It must influence requirements, validation, architecture, error handling, authentication, authorization, and publication practices. In Travlr Getaways, the enhancement uses authentication and authorization to protect administrative functions, validates data on both the client and server where appropriate, protects API operations, and avoids exposing internal errors to users. In the database enhancement, query allow-listing prevents clients from selecting arbitrary fields or operators, configuration values are separated from source code, and the data-access layer acts as the security boundary for query construction.

Security also guided how I prepared the public portfolio. Credentials, connection strings, and sensitive values must not be committed to a public repository. Published evidence should demonstrate the work without exposing secrets or personal information. Across the artifacts, I used malformed-input tests, invalid-reference tests, duplicate detection, safe error responses, strict compilation settings, and sanitizers to expose weaknesses before they affect users or data. These practices show that I now anticipate misuse and failure conditions instead of testing only the successful path.

How the Portfolio Artifacts Work Together

The three artifacts represent different categories, but together they demonstrate one consistent approach to software development. The Travlr Getaways enhancement shows how I design a full-stack workflow and connect user needs to frontend, API, authentication, and data concerns. The course-planning enhancement shows how I select and evaluate data structures and algorithms when dependency relationships determine the correctness of a result. The animal-rescue dashboard shows how database architecture, query controls, aggregation, indexing, and measurement affect the reliability and performance of an application.

Artifact	Primary Focus	Representative Evidence	Professional Value
CS 465 Travlr Getaways	Software design and engineering	Validated inquiry workflow, authentication, protected APIs, testing, and updated documentation	Demonstrates maintainable full-stack development centered on customer and administrator needs

Artifact	Primary Focus	Representative Evidence	Professional Value
CS 300 Course Planning Assistant	Algorithms and data structures	Graph model, hash-based lookup, topological sorting, validation scenarios, Big-O analysis, and sanitizer testing	Demonstrates correct dependency processing and explicit evaluation of efficiency and trade-offs
CS 340 Animal Rescue Dashboard	Databases	Server-side pagination, aggregation, allow-listed queries, deterministic sorting, automated tests, and index measurements	Demonstrates secure and measurable database-backed application design

Together, these artifacts demonstrate that I can evaluate an existing system, select an appropriate improvement, implement it, test it, document its behavior, and explain why the result matters. They also show that software engineering, algorithms, databases, communication, collaboration, and security are interconnected. A reliable application depends on all of them.

Readiness for the Profession

Completing this degree represents more than learning individual programming languages or tools. It reflects a long path from my first experiences with a home computer, through circumstances that delayed college, to discovering programming and choosing computer science as my profession. That path strengthened my persistence and willingness to keep learning. It also taught me that technical growth comes from examining mistakes, accepting feedback, and improving a solution until the evidence supports the result.

I am entering the next stage of my career with a stronger foundation in backend and full-stack development, data structures and algorithms, databases, testing, documentation, and secure design. I understand that tools and technologies will continue to change, so my most important ability is not familiarity with one framework. It is the ability to learn, evaluate a problem, communicate with the people affected by it, make defensible design choices, and verify that the implementation works. The work in this ePortfolio demonstrates those abilities and provides a foundation for continued growth as a software developer.